



Etablissement Inter – Etats d'Enseignement Supérieur
CENTRE D'EXCELLENCE TECHNOLOGIQUE PAUL BIYA
BP 13719 Yaoundé (Cameroun) Tél. (237) 22 72 99 58 - Fax (237) 22 72 99 57
Site web : www.iaicameroun.com E-mail : contact@iaicameroun.com

Cours Langage de Modélisation Unifié UML

Mme ANGA Orcelie
Ingénieur Informaticien Enseignante à l'IAI Cameroun

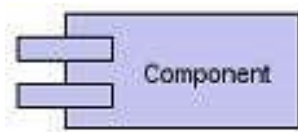
Diagramme des composants

Le diagramme des composants est principalement employé pour décrire les dépendances entre les divers composants logiciels tels que la dépendance entre les fichiers exécutables et les fichiers source.

Représentation : Ils décrivent les composants et leurs dépendances dans l'environnement de réalisation.

i. Composants

Un composant représente une entité logicielle d'un système. Un composant est représenté par une boîte rectangulaire, avec deux rectangles dépassant du côté gauche.



UML définit 5 stéréotypes aux composants :

- « document » : un document quelconque,
- « exécutable » : un programme qui peut s'exécuter,
- « fichier » : un document contenant un code source ou des données,
- « bibliothèque » : une bibliothèque statique ou dynamique,
- « table » : une table de base de données relationnelle.

ii. Dépendance

Une dépendance est utilisée pour modéliser la relation entre deux composants. La notation utilisée pour cette relation de dépendance est une flèche pointillée, se dirigeant d'un composant donné au composant dont il dépend. ----->

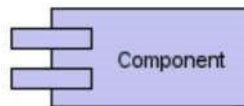
3. Diagramme de déploiement

Le diagramme de déploiement modélise les composants matériels utilisés pour implémenter un système et l'association entre ces composants. Des diagrammes de déploiement peuvent être mise en œuvre dès la phase de conception pour documenter l'architecture physique du système.

Représentation : Les éléments utilisés dans des diagrammes de déploiement sont des composants, comme dans les diagrammes des composants, et des nœuds, qui représentent les ressources physiques de traitement du système, et leurs associations.

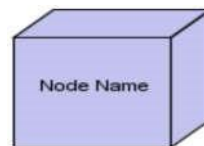
i. Composant

Un composant représente une entité logicielle du système. (Fichier de code source, programmes, documents, fichiers de ressource, etc.). Sur un diagramme de déploiement, les composants sont placés dans des nœuds pour identifier l'endroit de leur déploiement.



ii. Nœud

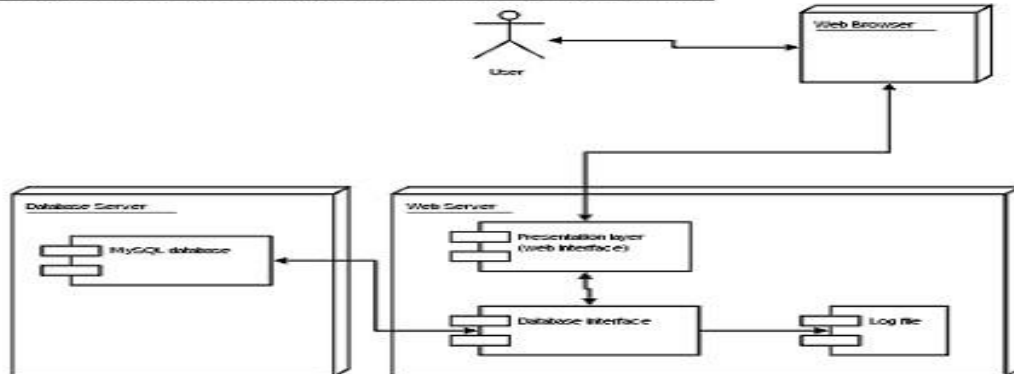
Un nœud représente un ensemble d'éléments matériels du système. Cette entité est représentée par un cube tridimensionnel.



iii. Association

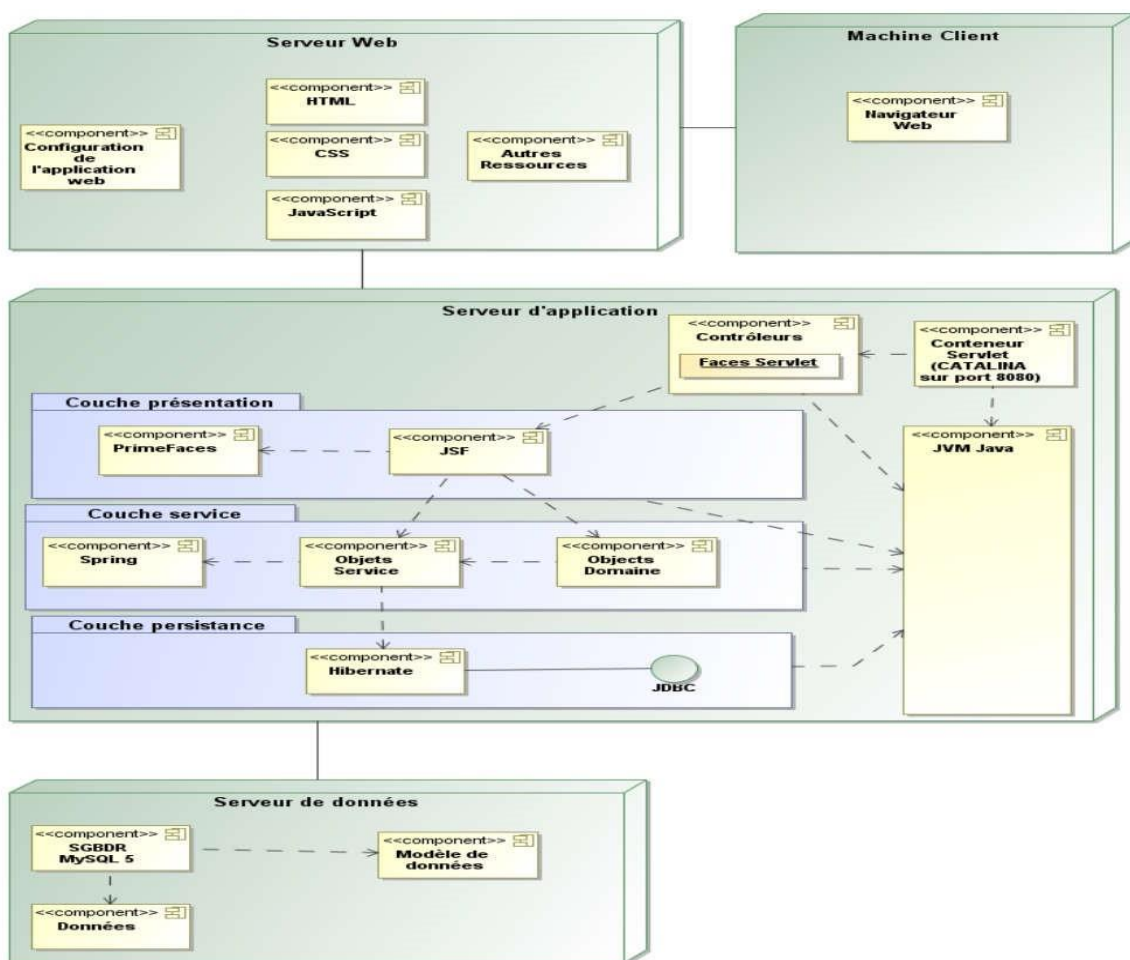
Une association, représentée par une ligne pleine entre deux nœuds, indique une ligne

Exemple de diagramme de déploiement pour une application web



de communication entre les éléments matériels.

Exemple2



II. Les vues dynamiques

Les cas d'utilisation sont centrés sur les besoins des utilisateurs. Ils aident à construire le bon système. Ils disent ce qu'un système doit faire, non comment il doit le faire. La vue des cas d'utilisation est une description fonctionnelle des besoins, structurée par rapport à des acteurs.

Le passage à l'approche objet s'effectue en associant une collaboration à chaque cas d'utilisation.

Une collaboration décrit les objets du domaine, les connexions entre ces objets et les messages échangés par les objets.

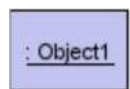
1. Le diagramme de séquence

Les diagrammes des séquences documentent les interactions à mettre en œuvre entre les classes pour réaliser un résultat, tel qu'un cas d'utilisation. UML étant conçu pour la programmation orientée objet, ces communications entre les classes sont reconnues comme des messages. Le diagramme des séquences énumère des objets horizontalement, et le temps verticalement. Il modélise l'exécution des différents messages en fonction du temps.

Représentation : Dans un diagramme des séquences, les classes et les acteurs sont énumérés en colonnes, avec leurs lignes de vie verticales indiquant la durée de vie de l'objet.

i. Objet

Les objets sont des instances des classes, et sont rangés horizontalement. La représentation graphique pour un objet est similaire à une classe (un rectangle) précédée du nom d'objet (facultatif) et des deux points (:).



ii. Acteur

Les acteurs peuvent également communiquer avec des objets, ainsi ils peuvent eux aussi être énumérés en colonne. Un acteur est modélisé en utilisant le symbole habituel :



iii. Ligne de vie

Les lignes de vie, LifeLine, identifient l'existence de l'objet par rapport au temps. La notation utilisée pour une ligne de vie est une ligne pointillée verticale partant de l'objet.



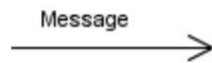
iv. Activation

Les activations, sont modélisées par des boîtes rectangulaires sur la ligne de vie. Elles indiquent quand l'objet effectue une action.

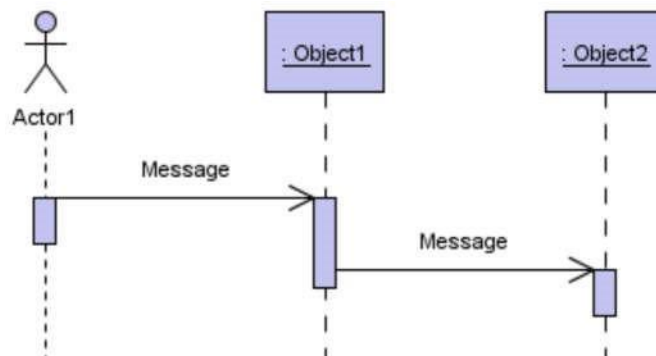


v. Message

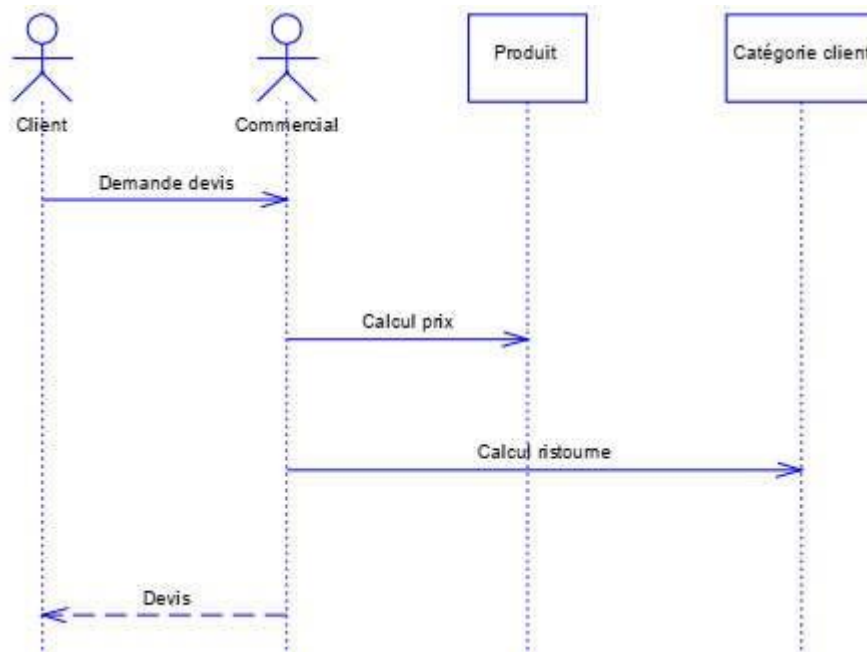
Les messages, modélisés par des flèches horizontales entre les activations, indiquent les communications entre les objets.



L'exemple ci-dessous représente un diagramme des séquences qui utilise des objets par défaut (aucun nom n'est spécifié).



Exemple



2. Le diagramme de collaboration

Les diagrammes de collaboration modélisent les interactions entre les objets. Ce type de diagramme est un croisement entre un diagramme des objets et un diagramme des séquences. À la différence du diagramme des séquences qui modélise l'interaction dans un format de type ligne-colonne, le diagramme des collaborations emploie une disposition libre des objets tels qu'on les trouve dans un diagramme des objets. Ceci facilite la vision de toutes les interactions impliquant un objet particulier.

Afin de maintenir l'ordonnancement des messages dans un tel diagramme de formes libres, ces derniers sont notés avec un numéro chronologique. La lecture d'un diagramme des collaborations implique de commencer au message 1.0, et de suivre ainsi la séquence des messages d'objet en objet.

Représentation :

i. Objet :

Les objets, instances des classes, représentent une des entités impliquées dans les communications. La représentation graphique pour un objet est similaire à une classe (un rectangle) précédée du nom d'objet (facultatif) et des deux points (:).



ii. Acteur

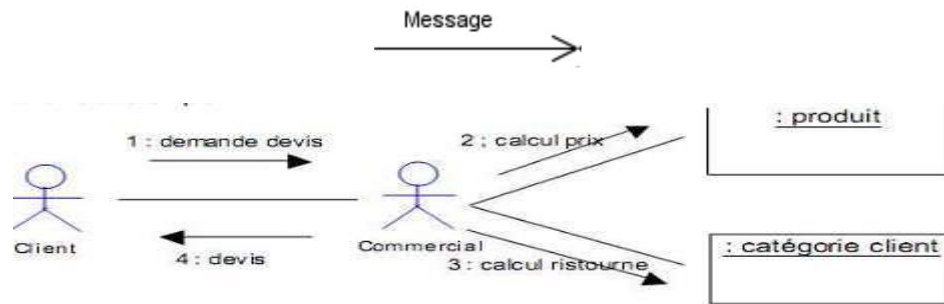
Les acteurs peuvent également communiquer avec des objets, aussi peuvent-ils être présents sur des diagrammes de collaborations. Un acteur est modélisé en utilisant le symbole habituel suivant :



iii. Message

Les messages, modélisés par des flèches entre les objets, sont affectés d'un numéro et indiquent les communications entre les objets.

Exemple



3. Le diagramme d'état transition

Les diagrammes d'état sont utilisés pour documenter les divers modes ("état") qu'une classe peut prendre, et les événements qui causent une transition d'état. Ils ont pour rôle de représenter les traitements (opérations) qui vont gérer le domaine étudié. Ils définissent l'enchaînement des états de classe et font donc apparaître l'ordonnancement des travaux.

Représentation :

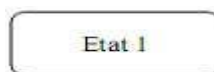
i. Etat :

Un état correspond à une situation durable dans laquelle se trouvent les objets d'une classe.

On lui associe les règles de gestion et les activités particulières.

Etat : objets d'une classe + règles de gestion + changements d'états

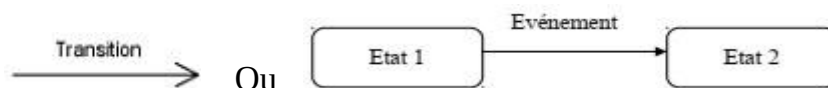
Elle est représentée par rectangle avec les coins arrondis, contenant le nom de l'état.



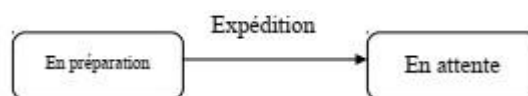
Exemples pour une commande : Etat "en préparation" et Etat "en cours" ii.

Transition ou événement

Une transition décrit le changement de l'état d'un objet, provoqué par un événement. Un objet passe d'un état à un autre suite à un événement, certains événements pouvant ne pas provoquer de changement d'état. Une transition est une relation entre 2 états. Elle est orientée. Sa représentation symbolique est une flèche sur laquelle est annoté l'événement qui concourt au changement d'état.



Exemple : Une commande passera dans l'état "En attente" dès lors qu'elle aura été expédiée.



iii. Etat initial

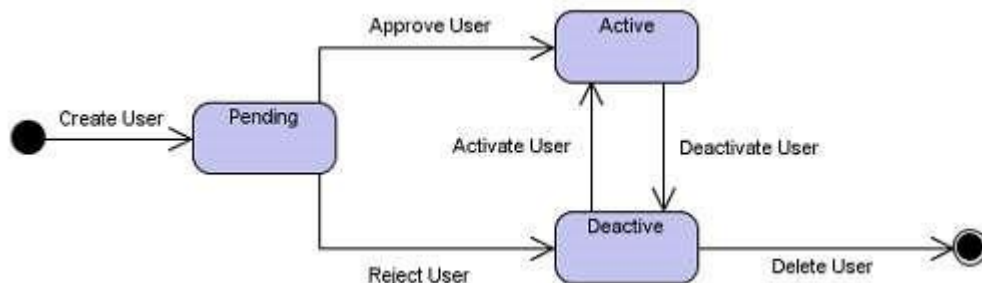
L'état initial est l'état d'un objet avant toutes transitions. L'état initial est représenté par un cercle plein. Un seul état initial est autorisé sur un diagramme. ●

iv. Etat final

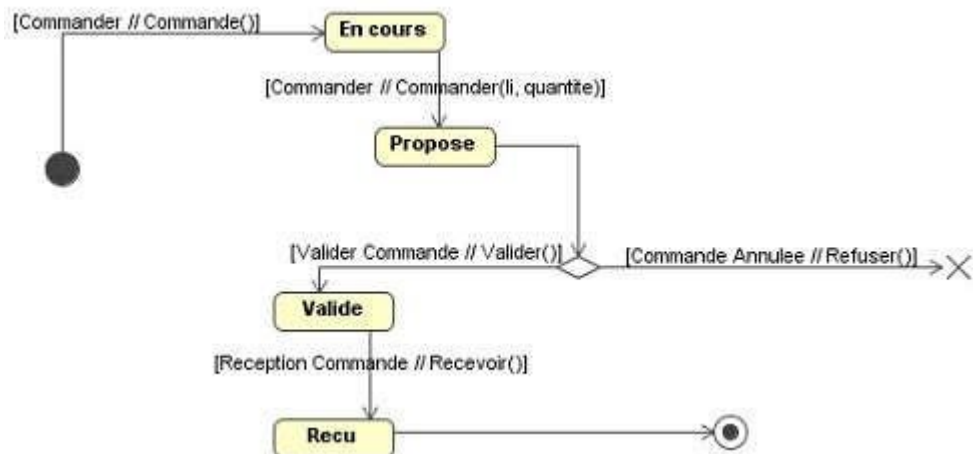
L'état final représente la destruction de l'objet que nous modélisons. Selon les événements, il peut exister plusieurs états finaux. Ces états sont représentés par un cercle plein entouré d'un cercle. ⊙



Exemple 1 : Diagramme d'état qui modélise l'état du compte d'un utilisateur :



Exemple 2 : de diagramme d'état qui modélise l'état d'une commande :



4. Le diagramme d'activité

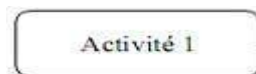
Les diagrammes d'activité sont utilisés pour documenter le déroulement des opérations dans un système, du niveau commercial au niveau opérationnel (de haut en bas). En regardant un diagramme d'activité, vous trouverez des éléments des diagrammes d'état. En fait, le diagramme d'activité est une variante du diagramme d'état où les "états" représentent des opérations, et les transitions représentent les activités qui se produisent quand l'opération est

terminée. L'usage général des diagrammes d'activité permet de faire apparaître les flots de traitements induits par les processus internes par rapport aux événements externes.

Représentation :

i. Etat d'activité ou une activité :

L'état d'activité marque une action faite par un objet. Il est représenté par un rectangle arrondi.



ii. Transition

Quand un état d'activité est accompli, le traitement passe à un autre état d'activité. Les transitions sont utilisées pour marquer ce passage. Les transitions sont modélisées par des flèches.



iii. Couloir

Le diagramme d'activités fait intervenir les acteurs de chaque activité. Chaque activité sera placée dans une colonne (couloir) qui correspond à l'acteur. Les objets sont énumérés audessus de la colonne, et les barres verticales séparent les colonnes pour former les couloirs d'activités.



iv. Etat initial

L'état initial marque le point d'entrée la première activité. Il est représenté, comme dans le diagramme d'état, par un cercle plein. Il ne peut y avoir qu'un seul état initial sur un diagramme. ●

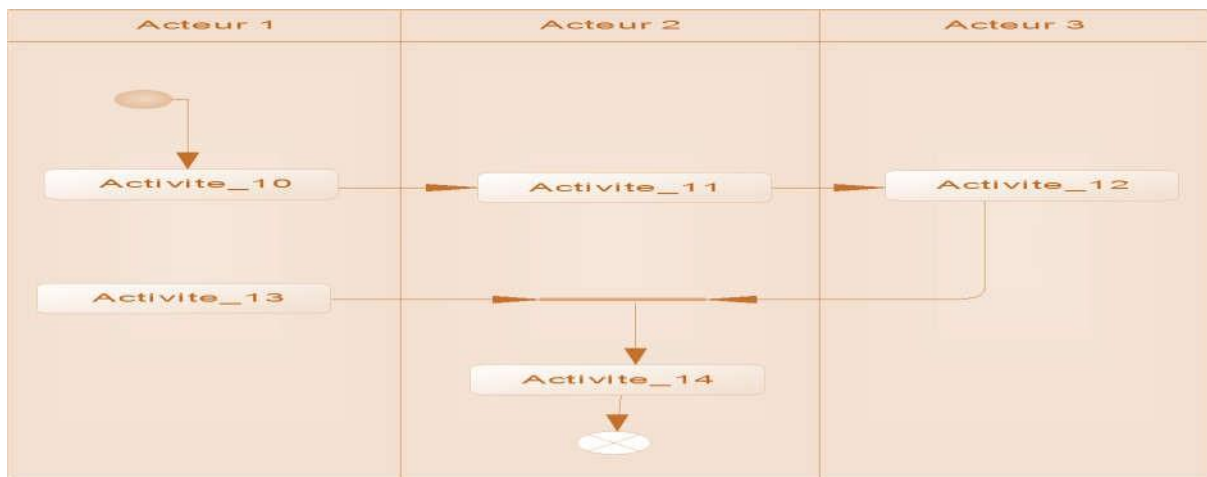
v. Etat final

L'état final marque la fin du déroulement des opérations modélisées. Il peut y avoir des états finaux multiples sur un diagramme. Ils sont représentés par un cercle plein entouré d'un autre cercle. ●

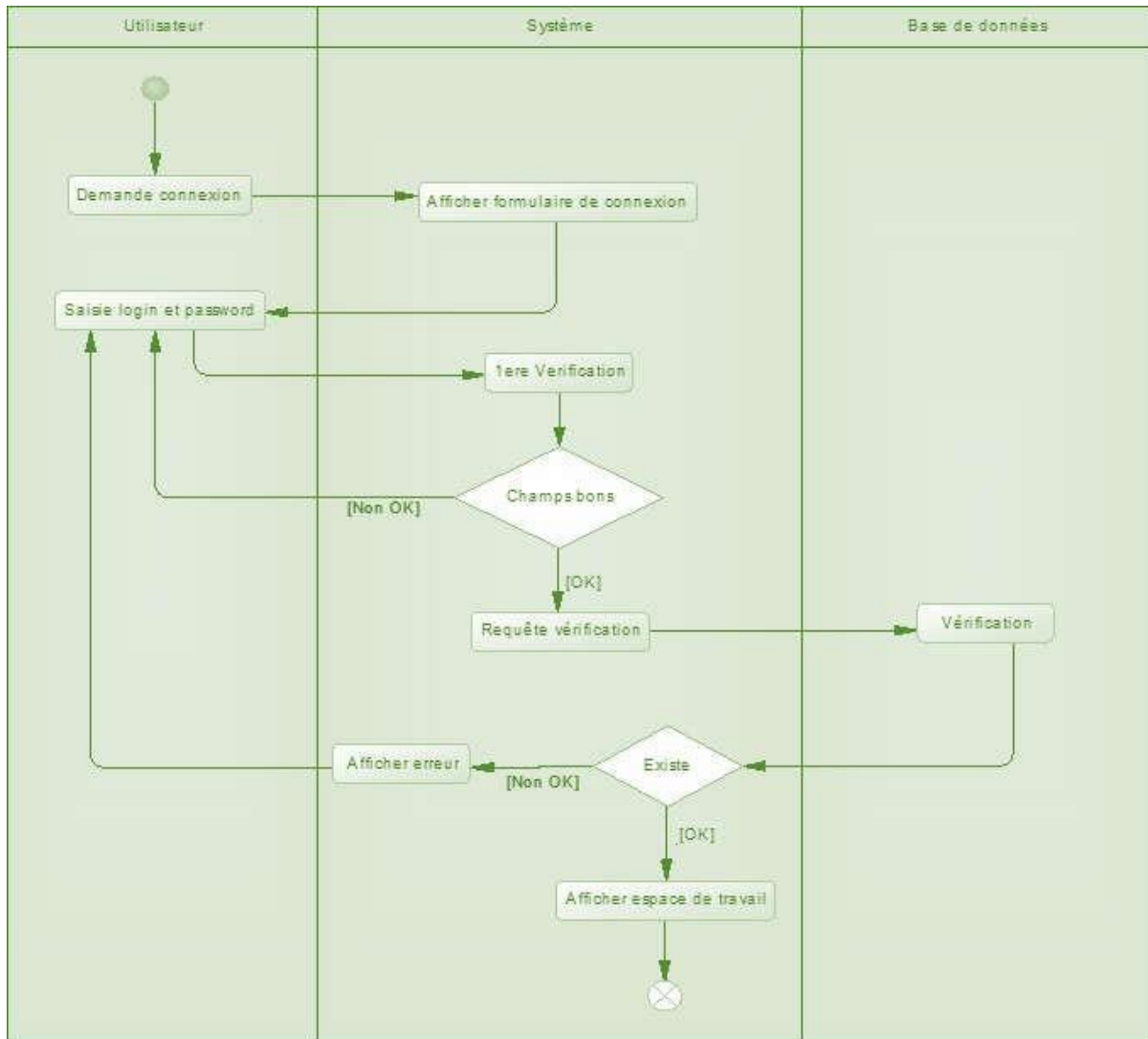
vi. Barre de Synchronisation

Souvent, certaines activités peuvent être faites en parallèle. Pour dédoubler le traitement "Fork", ou le reprendre quand des activités multiples ont été accomplies ("join"), des barres de synchronisation sont utilisées. Celles-ci sont modélisées par des rectangles pleins, avec des transitions multiples entrantes ou sortantes.

Formalisme du Diagramme



Exemple : Authentification d'un utilisateur



III. Modélisation et association des diagrammes

UML étant un langage de modélisation, n'impose pas une méthode de travail. Pour la réalisation d'un projet, l'utilisation de tous les diagrammes n'est pas obligatoire. Leur utilisation varie en fonction des exigences et des fonctionnalités du système étudié.

Le tableau ci-dessous résume l'utilisation de quelques diagrammes pour la réalisation de chaque phase d'un projet.

Besoins des utilisateurs : spécifier le système en définissant les acteurs, les cas d'utilisation et les associations équivalentes)	Diagramme de cas d'utilisation
---	--------------------------------

Interactions entre objets : Modéliser les objets communicants en identifiant et en nommant les instances (objets physiques puis abstraits), on spécifie les liens entre les objets et les messages transitant par ces liens.)	Diagramme de séquence Diagramme de collaboration
Structure statique : Modéliser la structure de l'application	Diagramme de classes Diagramme objet
Dynamique des objets : Modéliser le comportement des objets et les traitements.	Diagramme états -transition Diagramme d'activités
Réalisation et déploiement : Modéliser l'implantation de l'application	Diagramme de composants Diagramme de déploiement

IV. Les outils de modélisation

Voici une liste non exhaustive des logiciels de modélisation des diagrammes en UML.

Logiciels libres :

Neptune outil permettant de vérifier les modèles UML1.x via des règles OCL ;

ATL solution open source pour faire des transformations de modèles vers ou depuis UML (entre autres);

ArgoUml un modeleur UML sous Licence BSD ;

Gaphor un modeleur UML sous GPL;

BOUML, un modeleur UML sous GPL pour Windows, Linux, MacOS X et Solaris ;

Eclipse UML2, Méta modèle UML2, sans interface graphique ;

Netbeans [1], logiciel open source de Sun;

Papyrus un modeleur UML2 open source pour la plateforme Eclipse Licence EPL;

Delphia Object Modeler (version personnelle), Outil de modélisation et de prototypage.

Logiciels propriétaires

Enterprise Architect, un outil de modélisation UML ;

MagicDraw, un éditeur de diagrammes UML ;

Objectteering d'Objectteering Software ;

Poseidon (version commerciale), basé sur ArgoUml (logiciel libre);

PowerAMC / PowerDesigner [3], de Sybase (un outil de modélisation complet intégrant l'UML en plus des classiques MCD, MPD ...) ;

Rhapsody de Telelogic pour une modélisation PSM (Platform Specific Model) complète de systèmes ou de logiciels embarqués ;

Rational Software Architect/ Rational Software Modeler (et toujours Rose/XDE), de IBM Software Rational ;

SDE for Eclipse, un plugin UML pour Eclipse;

Visual Paradigm for UML, de Visual Paradigm International Ltd;

Delphia Object Modeler (version commerciale), Outil de modélisation et de prototypage. Diagrammes de classe et d'état. Langage objet intégré. Générateur de Java...

V. Quelques langages de programmation orientés objet

- Ada
- C++ (voir aussi la catégorie C++)
- C#
- Delphi (=Pascal orienté objet)
- Java
- Kylix
- Objective-C
- Objective Caml (ocaml)
- Perl
- PHP (Depuis la version 4)
- Python
- SmallTalk (totalement objet) Etc.

Chapitre IV : Le langage OCL et les processus unifiés

Partie I : Le processus UP

I. Définition du processus unifié UP

La maîtrise des processus de développement implique une organisation et un suivi des activités : c'est ce à quoi s'attachent les différentes méthodes qui s'appuient sur l'utilisation

du langage UML pour modéliser un système d'information. UP (Unified Process) est une méthode générique de développement de logiciel. Générique signifie qu'il est nécessaire d'adapter UP au contexte du projet, de l'équipe, du domaine et/ou de l'organisation (exemple : R.UP, X.UP, 2 TUP...).

1. Les caractéristiques d'UP

Le processus unifié est un processus de développement logiciel : il regroupe les activités à mener pour transformer les besoins d'un utilisateur en système logiciel.

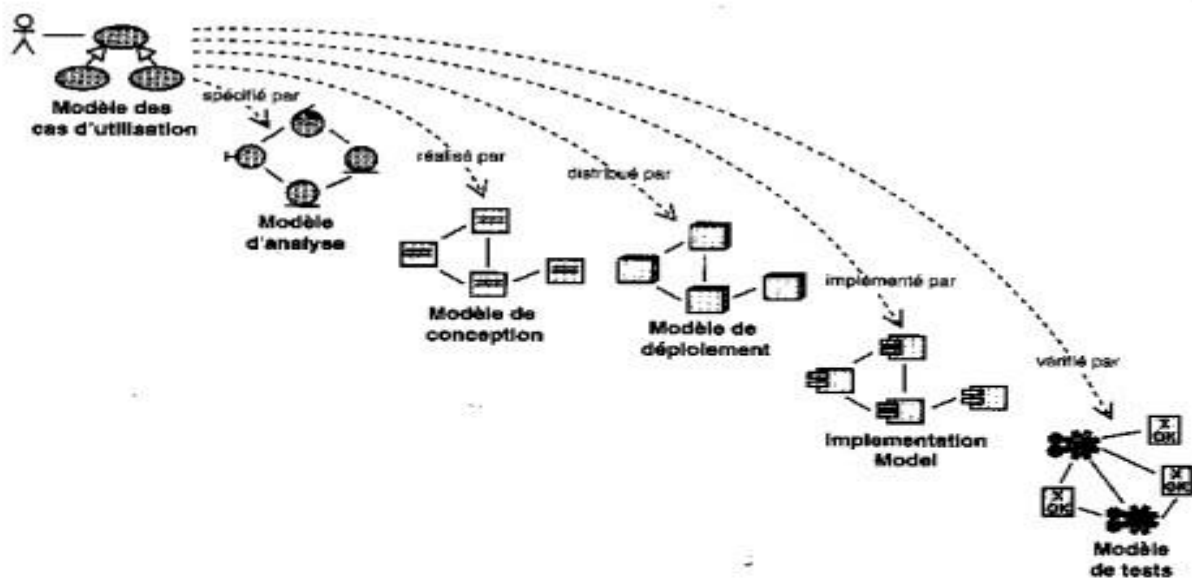
Les caractéristiques essentielles du processus unifié sont :

- Il est à base de composants,
- Il utilise le langage UML (ensemble d'outils et de diagramme),
- Il est piloté par les cas d'utilisation,
- Il est centré sur l'architecture, - Il est itératif et incrémental.

a) Le processus unifié est piloté par les cas d'utilisation

L'objectif principal d'un système logiciel est de rendre service à ses utilisateurs ; il faut par conséquent bien comprendre les désirs et les besoins des futurs utilisateurs. Le processus de développement sera donc centré sur l'utilisateur. Le terme utilisateur ne désigne pas seulement les utilisateurs humains mais également les autres systèmes.

Les cas d'utilisation ne sont pas un simple outil de spécification des besoins du système. Ils vont complètement guider le processus de développement à travers l'utilisation de modèles basés sur l'utilisation du langage UML.



- A partir du modèle des cas d'utilisation, les développeurs créent une série de modèles de conception et d'implémentation réalisant les cas d'utilisation.
- Chacun des modèles successifs est ensuite révisé pour en contrôler la conformité par rapport au modèle des cas d'utilisation.
- Enfin, les testeurs testent l'implémentation pour s'assurer que les composants du modèle d'implémentation mettent correctement en œuvre les cas d'utilisation.

b) Le processus unifié est centré sur l'architecture

Dès le démarrage du processus, on aura une vue sur l'architecture à mettre en place. L'architecture d'un système logiciel peut être décrite comme les différentes vues du système qui doit être construit. L'architecture logicielle équivaut aux aspects statiques et dynamiques les plus significatifs du système. L'architecture émerge des besoins de l'entreprise, tels qu'ils sont exprimés par les utilisateurs et autres intervenants et tels qu'ils sont reflétés par les cas d'utilisation.

Elle subit également l'influence d'autres facteurs :

- la plate-forme sur laquelle devra s'exécuter le système ;
- les briques de bases réutilisables disponibles pour le développement ;
- les considérations de déploiement, les systèmes existants et les besoins non fonctionnels (performance, fiabilité..).

c) Le processus unifié est itératif et incrémental

Le développement d'un produit logiciel destiné à la commercialisation est une vaste entreprise qui peut s'étendre sur plusieurs mois. On ne va pas tout développer d'un coup. On peut découper le travail en plusieurs parties qui sont autant de mini projets. Chacun d'entre eux représentant une itération qui donne lieu à un incrément.

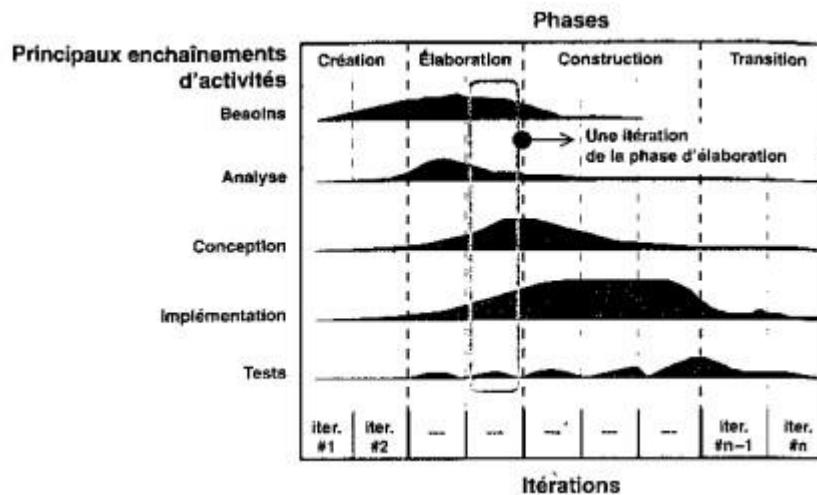
Une itération désigne la succession des étapes de l'enchaînement d'activités, tandis qu'un incrément correspond à une avancée dans les différents stades de développement. Le choix de ce qui doit être implémenté au cours d'une itération repose sur deux facteurs :

- Une itération prend en compte un certain nombre de cas d'utilisation qui ensemble, améliorent l'utilisabilité du produit à un certain stade de développement.
- L'itération traite en priorité les risques majeurs.

Un incrément constitue souvent un additif. A chaque itération, les développeurs identifient et spécifient les cas d'utilisations pertinents, créent une conception en se laissant guider par l'architecture choisie, implémentent cette conception sous forme de composants et vérifie que ceux-ci sont conformes aux cas d'utilisation. Dès qu'une itération répond aux objectifs fixés le développement passe à l'itération suivante. Pour rentabiliser le développement il faut sélectionner les itérations nécessaires pour atteindre les objectifs du projet. Ces itérations devront se succéder dans un ordre logique. Un projet réussi suivra un déroulement direct, établi dès le début par les développeurs et dont ils ne s'éloigneront que de façon très marginale. L'élimination des problèmes imprévus fait partie des objectifs de réduction des risques.

2. Le cycle de vie d'UP

Présentation



Phase	Description et Enchaînement d'activités
Phase de création	Traduit une idée en vision de produit fini et présente une étude de rentabilité pour ce produit - Que va faire le système pour les utilisateurs ? - A quoi peut ressembler l'architecture d'un tel système ? - Quels sont l'organisation et les coûts du développement de ce produit ? On fait apparaître les principaux cas d'utilisation. L'architecture est provisoire, identification des risques majeurs et planification de la phase d'élaboration.
Phase d'élaboration	Permet de préciser la plupart des cas d'utilisation et de concevoir l'architecture du système. L'architecture doit être exprimée sous forme de vue de chacun des modèles. Émergence d'une architecture de référence . A l'issue de cette phase, le chef de projet doit être en mesure de prévoir les activités et d'estimer les ressources nécessaires à l'achèvement du projet.
Phase de construction	Moment où l'on construit le produit. L'architecture de référence se métamorphose en produit complet, elle est maintenant stable. Le produit contient tous les cas d'utilisation que les chefs de projet, en accord avec les utilisateurs ont décidé de mettre au point pour cette version. Celle-ci doit encore avoir des anomalies qui peuvent être en partie résolues lors de la phase de transition.
Phase de transition	Le produit est en version bêta. Un groupe d'utilisateurs essaye le produit et détecte les anomalies et défauts. Cette phase suppose des activités comme la fabrication, la formation des utilisateurs clients, la mise en œuvre d'un service d'assistance et la correction des anomalies constatées. (où le report de leur correction à la version suivante)

Description

Le processus unifié répète un certain nombre de fois une série de cycles. Tout cycle se conclut par la livraison d'une version du produit aux clients et s'articule en 4 phases : création, élaboration, construction et transition, chacune d'entre elles se subdivisant à son tour en itérations. Pour mener efficacement le cycle, les développeurs ont besoin de construire toutes les représentations du produit logiciel :

Modèle des cas d'utilisation	Expose les cas d'utilisation et leurs relations avec les utilisateurs
Modèle d'analyse	Détaille les cas d'utilisation et procède à une première répartition du comportement du système entre divers objets
Modèle de conception	Définit la structure statique du système sous forme de sous système, classes et interfaces ; Définit les cas d'utilisation réalisés sous forme de collaborations entre les sous systèmes les classes et les interfaces
Modèle d'implémentation	Intègre les composants (code source) et la correspondance entre les classes et les composants
Modèle de déploiement	Définit les nœuds physiques des ordinateurs et l'affectation de ces composants sur ces nœuds.
Modèle de test	Décrit les cas de test vérifiant les cas d'utilisation
Représentation de l'architecture	Description de l'architecture

3. Exemple de processus de développement logiciel qui implémente le Processus Unifié : 2TUP (Two tracks unified process)

Dans le processus 2TUP, les activités de développement sont organisées suivant 5 workflows qui décrivent :

- La capture des besoins,
- L'analyse,
- La conception, ➤ L'implémentation ➤ Et le test.

Le processus 2TUP est une trame des meilleures pratiques de développement, il doit être utilisé comme un guide pour réaliser un projet et non comme l'arme ultime et universelle de développement.

A. Le cycle de développement avec 2TUP

Le 2TUP propose un cycle de développement en Y, qui dissocie les aspects techniques des aspects fonctionnels. Il commence par une étude préliminaire qui consiste essentiellement à identifier les acteurs qui vont interagir avec le système à construire, les messages qu'échangent les acteurs et le système. En suite à produire le cahier des charges et à modéliser le contexte. Le processus s'articule autour de 3 phases essentielles : une branche technique ; une branche fonctionnelle et une phase de réalisation.

i. La branche fonctionnelle

La branche fonctionnelle capitalise la connaissance du métier de l'entreprise. Cette branche comporte :

- La capture des besoins fonctionnels, qui produit un modèle des besoins focalisé sur le métier des utilisateurs. Elle qualifie au plus tôt le risque de produire un système inadapté aux utilisateurs. De son côté, la maîtrise d'œuvre consolide les spécifications et en vérifie la cohérence et l'exhaustivité.
- L'analyse, qui consiste à étudier précisément la spécification fonctionnelle de manière à obtenir une idée de ce que va réaliser le système en termes de métier. Les résultats de l'analyse ne dépendent d'aucune technologie particulière.

ii. La branche technique

La branche technique capitalise un savoir-faire technique et/ou des contraintes techniques. Les techniques développées pour le système le sont indépendamment des fonctions à réaliser. Cette branche droite comporte :

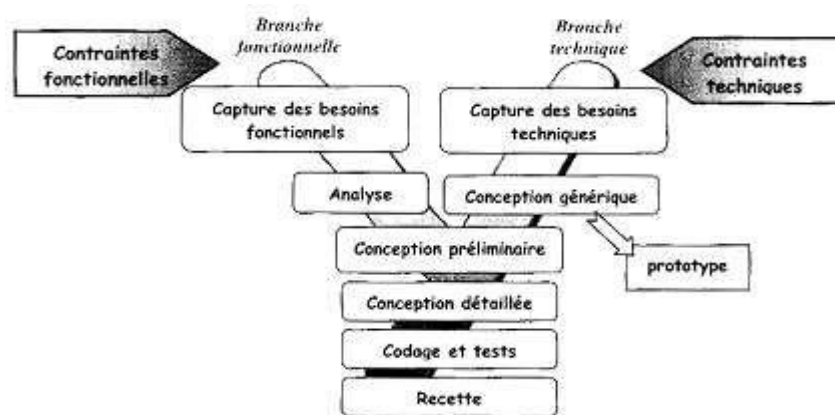
- La capture des besoins techniques, qui recense toutes les contraintes et les choix dimensionnant la conception du système. Les outils et les matériels sélectionnés ainsi que la prise en compte de contraintes d'intégration avec l'existant conditionnent généralement des pré-requis d'architecture technique ;
- La conception générique, qui définit ensuite les composants nécessaires à la construction de l'architecture technique. Cette conception est complètement indépendante des aspects fonctionnels. Elle a pour objectif d'uniformiser et de réutiliser les mêmes mécanismes pour tout un système. L'architecture technique construit le squelette du système informatique et écarte la plupart des risques de niveau technique. L'importance de sa réussite est telle qu'il est conseillé de réaliser un prototype pour assurer sa validité.

iii. La phase de réalisation

La phase de réalisation consiste à réunir les deux branches, permettant de mener une conception applicative et enfin la livraison d'une solution adaptée aux besoins. La branche du milieu comporte :

- La conception préliminaire, qui représente une étape délicate, car elle intègre le modèle d'analyse dans l'architecture technique de manière à tracer la cartographie des composants du système à développer,
- La conception détaillée, qui étudie ensuite comment réaliser chaque composant ;
- L'étape de codage, qui produit ces composants et teste au fur et à mesure les unités de code réalisées,
- L'étape de recette, qui consiste enfin à valider les fonctions du système développé.

iv. Représentation



4. Eléments de comparaisons entre MERISE et UML

<u>MERISE :</u>	<u>UML :</u>
Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprises	Unified Modeling Language
MERISE est une méthode systémique d'analyse et de conception de systèmes d'information. C'est-à-dire qu'elle utilise une approche systémique.	UML n'est cependant pas une méthode mais plutôt un langage de modélisation objet à qui il faut associer une démarche pour en faire une méthode. c'est le cas de la méthode 2TUP, RUT et XP.

MERISE propose de considérer le système réel selon deux points de vue: - Une vue statique (données) - Une vue dynamique (traitements). C'est-à-dire qu'avec la méthode MERISE, nous avons une étude séparée des données et des traitements.	UML propose une approche différente de celle de MERISE en ce sens qu'il associe les données et les traitements. Car avec UML, centraliser les données d'un type et les traitements associés permet de limiter les points de maintenance dans le code et facilite l'accès à l'information en cas d'évolution du logiciel. De plus, UML décrit la dynamique du système d'information comme un ensemble d'opérations attachées aux objets du système.
---	--

Partie II : Le langage OCL (Object Constraint Language)

Introduction

Pour spécifier complètement une application, les diagrammes UML seuls sont généralement insuffisants. Il est donc nécessaire de rajouter des contraintes.

C'est avec OCL (Object Constraint Language) qu'UML formalise l'expression des contraintes. Il s'agit donc d'un langage formel d'expression de contraintes bien adapté aux diagrammes d'UML, et en particulier au diagramme de classes.

OCL existe depuis la version 1.1 d'UML et est une contribution d'IBM. OCL fait partie intégrante de la norme UML depuis la version 1.3 d'UML. Dans le cadre d'UML 2.0, les spécifications du langage OCL figurent dans un document indépendant de la norme d'UML, décrivant en détail la syntaxe formelle et la façon d'utiliser ce langage.

OCL peut s'appliquer sur la plupart des diagrammes d'UML et permet de spécifier des contraintes sur l'état d'un objet ou d'un ensemble d'objets comme :

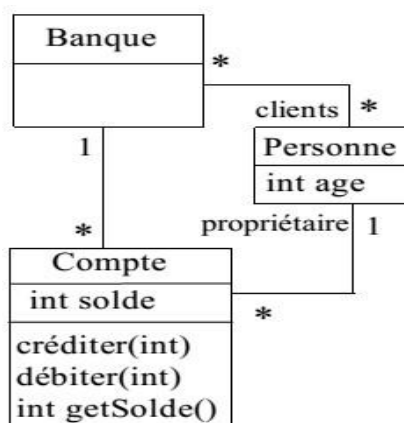
- des invariants sur des classes ;

- des pré-conditions et des post-conditions à l'exécution d'opérations : les préconditions doivent être vérifiées avant l'exécution et les post-conditions doivent être vérifiées après l'exécution ;
- des gardes sur des transitions de diagrammes d'états-transitions ou des messages de diagrammes d'interaction ;
- des ensembles d'objets destinataires pour un envoi de message ;
- des attributs dérivés, etc.

Note : Une expression OCL décrit une contrainte à respecter et pas le « code » d'une méthode.

I. Les principaux concepts d'OCL

Considérons le diagramme de classe suivant associé à une application de gestion bancaire.



1. Contexte (context)

Une contrainte est toujours associée à un élément de modèle. C'est cet élément qui constitue le contexte de la contrainte. On dit aussi qu'une expression OCL est toujours définie dans un contexte.

Il existe deux manières pour spécifier le contexte d'une contrainte OCL :

- En écrivant la contrainte entre accolades (**{}**) dans une note. L'élément pointé par la note est alors le contexte de la contrainte.
- En utilisant le mot-clef **context** dans un document accompagnant le diagramme.

Syntaxe : **context** <élément>

<élément> peut être une classe, une opération, etc.

Pour faire référence à un élément op (comme un opération) d'un classeur C (comme une classe), ou d'un paquetage, . . . , il faut utiliser les :: comme séparateur (comme C::op).

Exemple

- context Compte est le contexte est la classe Compte ;
- context Compte::getSolde() est le contexte de l'opération getSolde() de la classe Compte ;
- on peut simplement ajouter une note pour préciser contrainte : {solde > 0},

2. Invariants (inv)

Un invariant exprime une contrainte prédicative sur un objet, ou un groupe d'objets, qui doit être respectée en permanence. Syntaxe : inv : <expression_logique>

<expression_logique> est une expression logique qui doit toujours être vraie. Exemple

Le solde d'un compte doit toujours être positif.

context Compte

inv : solde > 0

3. Préconditions et postconditions (pre, post)

Une précondition (respectivement une postcondition) permet de spécifier une contrainte prédicative qui doit être vérifiée avant (respectivement après) l'appel d'une opération.

Dans l'expression de la contrainte de la postcondition, deux éléments particuliers sont utilisables :

- l'attribut result qui désigne la valeur retournée par l'opération,
- et <nom_attribut>@pre qui désigne la valeur de l'attribut <nom_attribut> avant l'appel de l'opération.

Syntaxe :

- Précondition : pre : <expression_logique>
- Postcondition : post : <expression_logique>

<expression_logique> est une expression logique qui doit toujours être vraie.

Exemple

Concernant la méthode débiter de la classe Compte, la somme à débiter doit être positive pour que l'appel de l'opération soit valide et, après l'exécution de l'opération, l'attribut solde doit avoir pour valeur la différence de sa valeur avant l'appel et de la somme passée en paramètre.

```
context Compte::débiter(somme : Real) pre
: somme > 0
post : solde = solde@pre - somme
```

Le résultat de l'appel de l'opération getSolde doit être égal à l'attribut solde.

```
context Compte : getSolde() : Real
post : result = solde
```

4. Types et opérations utilisables dans les expressions OCL

Le langage OCL possède un certain nombre de types prédéfinis et d'opérations prédéfinies sur ces types. Ces types et ces opérations sont utilisables dans n'importe quelle contrainte et sont indépendants du modèle auquel sont rattachées ces contraintes.

Le tableau suivant donne un aperçu des types et opérations prédéfinis dans les contraintes OCL.

Type	Exemples de valeurs	Opérateurs
Boolean	<i>true ; false</i>	<i>and ; or ; xor ; not ; implies ; if-then-else-endif ; ...</i>
Integer	<i>1 ; -5 ; 2 ; 34 ; 26524 ; ...</i>	<i>* ; + ; - ; / ; abs() ; ...</i>
Real	<i>1,5 ; 3,14 ; ...</i>	<i>* ; + ; - ; / ; abs() ; floor() ; ...</i>
String	<i>"To be or not to be ..."</i>	<i>concat() ; size() ; substring() ; ...</i>

OCL définit également la notion d'ensemble sous le terme générique de collection (collection en anglais). Il existe plusieurs sous-types du type abstrait Collection :

- Ensemble (Set) : collection non ordonnée d'éléments uniques (i.e. pas d'élément en double).
- Ensemble ordonné (OrderedSet) : collection ordonnée d'éléments uniques.
- Sac (Bag) : collection non ordonnée d'éléments identifiables (i.e. comme un ensemble, mais pouvant comporter des doublons).
- Séquence (Sequence) : collection ordonnée d'éléments identifiables.

Jusqu'à UML 2.0 exclu, les collections étaient toujours plates : une collection ne pouvait pas posséder des collections comme éléments. Cette restriction n'existe plus à partir d'UML 2.0.

5. Accès aux caractéristiques et aux objets dans les contraintes OCL

Dans une contrainte OCL associée à un objet, il est possible d'accéder aux caractéristiques (attributs, opérations et terminaison d'association) de cet objet, et donc, d'accéder de manière transitive à tous les objets (et leurs caractéristiques) avec qui il est en relation. ➤ Accès aux attributs et aux opérations (self)

Pour faire référence à un attribut ou une opération de l'objet désigné par le contexte, il suffit d'utiliser le nom de cet élément. L'objet désigné par le contexte est également accessible par l'expression self. On peut donc également utiliser la notation pointée : self.<propriété>.

Une opération peut avoir des paramètres, il faut alors les préciser entre les parenthèses de l'opération.

Par exemple, dans le contexte de la classe Compte, on peut utiliser les expressions suivantes :

- solde
- self.solde
- getSolde()
- self.getSolde()
- débiter(1000)
- self.débiter(1000)

Dans l'exemple précédent, le résultat de l'expression self.débiter(1000) est un singleton du type Real.

➤ Navigation via une association

Pour faire référence à un objet, ou un groupe d'objets, en association avec l'objet désigné par le contexte, il suffit d'utiliser le nom de la classe associée (en minuscule) ou le nom du rôle d'association du côté de cette classe. Quand c'est possible, il est préférable d'utiliser le nom de rôle de l'association du côté de l'objet auquel on désire faire référence.

Par exemple, dans le contexte de la classe Société (context Société) :

- directeur désigne le directeur de la société (résultat de type Personne) ;
- employé désigne l'ensemble des employés de la société (résultat de type Set(Personne)) ;

- employé.compte désigne l'ensemble des comptes de tous les employés de la société (résultat de type Bag(Compte)) ;
- employé.date_de_naissance désigne l'ensemble des dates de naissance des employés de la société (résultat de type Bag(Date)).

6. Illustration par l'exemple

Mise en situation

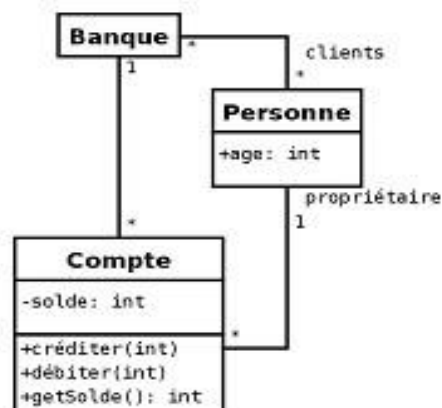
Plaçons-nous dans le contexte d'une application bancaire. Il nous faut donc gérer :

- des comptes bancaires,
- des clients,
- et des banques.

De plus, on aimerait intégrer les contraintes suivantes dans notre modèle :

- un compte doit avoir un solde toujours positif ;
- un client peut posséder plusieurs comptes ;
- une personne peut être cliente de plusieurs banques ;
- un client d'une banque possède au moins un compte dans cette banque ;
- un compte appartient forcément à un client ;
- une banque gère plusieurs comptes ;
- une banque possède plusieurs clients.

Diagramme de classes



Utilisation du langage de contrainte OCL sur l'exemple bancaire. On obtient

